

# Guia de reciclaje de componentes web

## Documento 1 - Funcionalidades 1 a 10 en el orden de la tabla de contenido original

Este documento corrige y reorganiza la guía para seguir exactamente el orden de la tabla de contenido del recurso unificado: Navbar fija glass, Sidebar ocultable, Submenus laterales, Router de secciones, Dropdown superior, Tema claro/oscuro, Scroll progress, Back to top, Reveal on scroll y Contador animado.

La idea no es solo ver el código, sino aprender a reciclar cada funcionalidad en cualquier proyecto. Por eso cada componente incluye: para que sirve, que copiar, que cambiar, código HTML, CSS, JavaScript modular, JavaScript normal, pasos de implementación, errores comunes y checklist.

## Convenciones para usar esta guía

| Convencion           | Significado                                                               | Ejemplo                         |
|----------------------|---------------------------------------------------------------------------|---------------------------------|
| ID                   | Identificador unico. No debe repetirse en la pagina.                      | id="sidebar"                    |
| Clase                | Nombre reutilizable para aplicar CSS o seleccionar elementos.             | class="glass-navbar"            |
| data-*               | Atributo personalizado para guardar datos que JS puede leer.              | data-section="inicio"           |
| is-open / is-visible | Clases de estado. JS las agrega o quita para cambiar la interfaz.         | .sidebar.is-open                |
| Modular              | Código con export/import. Recomendado para proyectos organizados.         | export class SidebarToggle {}   |
| Normal               | Código sin módulos. Util cuando quieres un solo archivo script-normal.js. | function initSidebarToggle() {} |

## Orden corregido de funcionalidades 1 a 10

| # | Funcionalidad       | Para que sirve                                | Archivo principal que toca |
|---|---------------------|-----------------------------------------------|----------------------------|
| 1 | Navbar fija glass   | Barra superior fija con blur y transparencia. | HTML + CSS + JS            |
| 2 | Sidebar ocultable   | Menu lateral que se abre y cierra.            | HTML + CSS + JS            |
| 3 | Submenus laterales  | Opciones internas desplegables.               | HTML + CSS + JS            |
| 4 | Router de secciones | Muestra una seccion diferente en el body.     | HTML + JS                  |
| 5 | Dropdown superior   | Menu desplegable en navbar.                   | HTML + CSS + JS            |
| 6 | Tema claro/oscuro   | Cambia variables CSS y guarda preferencia.    | CSS + JS                   |
| 7 | Scroll progress     | Barra que muestra avance de scroll.           | HTML + CSS + JS            |
| 8 | Back to top         | Boton para volver arriba.                     | HTML + CSS + JS            |

| #  | Funcionalidad    | Para que sirve                       | Archivo principal que toca |
|----|------------------|--------------------------------------|----------------------------|
| 9  | Reveal on scroll | Animacion al aparecer en pantalla.   | CSS + JS                   |
| 10 | Contador animado | Cuenta de 0 hasta un valor objetivo. | HTML + JS                  |

## Estructura recomendada de archivos

Para reciclar cualquier componente en proyectos nuevos, mantén una estructura simple. Puedes usar modulos o version normal, pero no mezcles las dos al mismo tiempo en la misma pagina para evitar duplicar eventos.

### Estructura sugerida

```
mi-proyecto/
├─ index.html
├─ styles.css
├─ funciones.js      // clases y funciones exportadas
├─ script.js         // importa y arranca componentes
└─ script-normal.js  // alternativa sin import/export
```

Regla practica: si usas import/export, en HTML carga `<script type="module" src="script.js"></script>`. Si no usas modulos, carga solo `<script src="script-normal.js"></script>`.

# 1. Navbar fija glass

La navbar fija glass es una barra superior que permanece visible mientras el usuario hace scroll. Usa transparencia, blur y sombra para verse cristalina sin tapar totalmente el fondo. Es muy usada en landing pages, dashboards, portafolios, ecommerce y aplicaciones web modernas.

Donde se usa en paginas reales: cabeceras de paginas, menus principales, paneles administrativos, sitios de marca, portafolios y single page applications.

Dependencias internas: puede trabajar sola, pero normalmente se conecta con el Router de secciones mediante botones con data-section.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

| Parte      | Que debes copiar                                        | Que puedes cambiar sin romperlo                                            |
|------------|---------------------------------------------------------|----------------------------------------------------------------------------|
| HTML       | Estructura visible del componente y atributos data-*.   | Textos, iconos, rutas, nombres de secciones, opciones y labels.            |
| CSS        | Clases del componente y, si aplica, variables de color. | Colores, blur, radios, espacios, sombras, tiempos de transicion.           |
| JS modular | Clase exportada con constructor e inicializacion.       | Selectores, nombres de clase activa, claves localStorage y comportamiento. |
| JS normal  | Funcion inicializadora sin import/export.               | Selectores y condiciones, igual que en la version modular.                 |

## Codigo HTML minimo

### HTML que debes llevar a index.html

```
<header class="glass-navbar" id="mainNavbar">
  <a href="#inicio" class="glass-navbar__brand">Mi Proyecto</a>

  <nav class="glass-navbar__links" aria-label="Menu principal">
    <button type="button" data-section="inicio">Inicio</button>
    <button type="button" data-section="servicios">Servicios</button>
    <button type="button" data-section="contacto">Contacto</button>
  </nav>

  <button type="button" class="glass-navbar__menu" id="openSidebar">
    Menu
  </button>
</header>
```

## Codigo CSS minimo

### CSS que debes llevar a styles.css

```

/* Variable util para que el contenido no quede debajo de la navbar */
:root {
  --navbar-height: 72px;
  --glass-bg: rgba(255, 255, 255, 0.14);
  --glass-border: rgba(255, 255, 255, 0.28);
  --primary: #2563eb;
}

body {
  padding-top: var(--navbar-height);
}

.glass-navbar {
  position: fixed;
  top: 0;
  left: 0;
  z-index: 1000;
  width: 100%;
  min-height: var(--navbar-height);
  display: flex;
  align-items: center;
  justify-content: space-between;
  gap: 16px;
  padding: 14px 24px;
  background: var(--glass-bg);
  border-bottom: 1px solid var(--glass-border);
  backdrop-filter: blur(18px);
  -webkit-backdrop-filter: blur(18px);
  box-shadow: 0 12px 30px rgba(15, 23, 42, 0.18);
}

.glass-navbar__brand {
  color: #ffffff;
  font-weight: 800;
  text-decoration: none;
}

.glass-navbar__links {
  display: flex;
  gap: 10px;
}

.glass-navbar button {
  border: 0;
  border-radius: 999px;
  padding: 10px 14px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.12);
  cursor: pointer;
}

.glass-navbar button:hover,
.glass-navbar button.is-active {
  background: var(--primary);
}

@media (max-width: 720px) {
  .glass-navbar__links {
    display: none;
  }
}

```

## JavaScript modular con export/import

### funciones.js - clase o funcion exportada

```
export class GlassNavbar {
  constructor(navbarId = "mainNavbar") {
    this.navbar = document.getElementById(navbarId);
    this.buttons = this.navbar?.querySelectorAll("[data-section]") || [];
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => {
        this.setActive(button);
      });
    });
  }

  setActive(activeButton) {
    this.buttons.forEach((button) => button.classList.remove("is-active"));
    activeButton.classList.add("is-active");
  }
}
```

## script.js - como llamarlo

```
// script.js
import { GlassNavbar } from './funciones.js';

new GlassNavbar();
```

## JavaScript normal sin modulos

### script-normal.js - version directa

```
function initGlassNavbar(navbarId = "mainNavbar") {
  const navbar = document.getElementById(navbarId);
  if (!navbar) return;

  const buttons = navbar.querySelectorAll("[data-section]");

  buttons.forEach((button) => {
    button.addEventListener("click", () => {
      buttons.forEach((item) => item.classList.remove("is-active"));
      button.classList.add("is-active");
    });
  });
}

initGlassNavbar();
```

## Variables, clases, IDs y atributos que debes cambiar

| Elemento        | Donde esta | Como adaptarlo                                                                |
|-----------------|------------|-------------------------------------------------------------------------------|
| #mainNavbar     | HTML y JS  | Cambia el id solo si tambien cambias el argumento new GlassNavbar('nuevold'). |
| .glass-navbar   | CSS y HTML | Puedes renombrarla, pero cambia todas sus reglas CSS.                         |
| data-section    | Botones    | Debe coincidir con las secciones si usas router.                              |
| --navbar-height | :root      | Ajusta la altura si tu navbar es mas alta o mas baja.                         |

| Elemento   | Donde esta | Como adaptarlo                                                           |
|------------|------------|--------------------------------------------------------------------------|
| --glass-bg | :root      | Sube o baja la transparencia; 0.10 es mas transparente, 0.30 mas solido. |

## Paso a paso para implementarlo en otro proyecto

1. Copia el bloque <header> justo despues de abrir <body>.
2. Copia las variables CSS y las reglas de .glass-navbar en tu styles.css.
3. Verifica que el body tenga padding-top para que el contenido no quede escondido debajo de la navbar.
4. Si vas a controlar secciones, pon data-section en cada boton y crea secciones con esos mismos nombres.
5. Importa GlassNavbar en script.js y crea new GlassNavbar().
6. Prueba scroll: la barra debe quedarse fija y el blur debe verse sobre el fondo.

## Errores comunes y como corregirlos

| Error                       | Causa probable                                                     | Solucion                                                      |
|-----------------------------|--------------------------------------------------------------------|---------------------------------------------------------------|
| La navbar tapa el contenido | Falta padding-top en body                                          | Usa body { padding-top: var(--navbar-height); }               |
| No se ve el blur            | El fondo detras es plano o el navegador no soporta backdrop-filter | Usa un fondo con degradados y agrega -webkit-backdrop-filter. |
| Los botones no se activan   | El JS no encuentra #mainNavbar                                     | Revisa que el id del HTML coincida con el constructor.        |

## Checklist de prueba

- La navbar queda fija al hacer scroll.
- Los botones cambian a estado activo al hacer clic.
- En celular se ocultan los links si asi lo definiste.
- El contenido no queda debajo de la barra superior.

## Adaptaciones profesionales recomendadas

- Agrega aria-label al nav para mejorar accesibilidad.
- Usa una clase .is-scrolled para oscurecer la navbar cuando el usuario baja.
- Conecta el boton Menu con el Sidebar ocultable de la funcionalidad 2.

## 2. Sidebar ocultable

El sidebar ocultable es un menu lateral que entra y sale de la pantalla cuando el usuario lo pide. Es muy comun en dashboards, paneles administrativos, aplicaciones tipo CRM, proyectos con muchas secciones y paginas responsive.

Donde se usa en paginas reales: dashboards, aplicaciones internas, ecommerce con filtros, sistemas de aprendizaje, paneles con muchas opciones.

Dependencias internas: puede abrirse desde un boton de la navbar; se complementa con overlay y submenus laterales.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

| Parte      | Que debes copiar                                        | Que puedes cambiar sin romperlo                                            |
|------------|---------------------------------------------------------|----------------------------------------------------------------------------|
| HTML       | Estructura visible del componente y atributos data-*.   | Textos, iconos, rutas, nombres de secciones, opciones y labels.            |
| CSS        | Clases del componente y, si aplica, variables de color. | Colores, blur, radios, espacios, sombras, tiempos de transicion.           |
| JS modular | Clase exportada con constructor e inicializacion.       | Selectores, nombres de clase activa, claves localStorage y comportamiento. |
| JS normal  | Funcion inicializadora sin import/export.               | Selectores y condiciones, igual que en la version modular.                 |

### Codigo HTML minimo

#### HTML que debes llevar a index.html

```
<button type="button" id="openSidebar" class="btn-menu">Abrir menu</button>

<aside class="sidebar" id="sidebar" aria-hidden="true">
  <div class="sidebar__header">
    <h2>Panel</h2>
    <button type="button" id="closeSidebar">Cerrar</button>
  </div>

  <nav class="sidebar__nav">
    <button type="button" data-section="inicio">Inicio</button>
    <button type="button" data-section="perfil">Perfil</button>
    <button type="button" data-section="configuracion">Configuracion</button>
  </nav>
</aside>

<div class="sidebar-overlay" id="sidebarOverlay"></div>
```

### Codigo CSS minimo

#### CSS que debes llevar a styles.css

```

body.sidebar-open {
  overflow: hidden;
}

.sidebar {
  position: fixed;
  top: 0;
  left: 0;
  z-index: 1200;
  width: min(320px, 86vw);
  height: 100vh;
  padding: 20px;
  color: #ffffff;
  background: rgba(15, 23, 42, 0.88);
  border-right: 1px solid rgba(255, 255, 255, 0.18);
  backdrop-filter: blur(20px);
  -webkit-backdrop-filter: blur(20px);
  transform: translateX(-105%);
  transition: transform 0.25s ease;
}

.sidebar.is-open {
  transform: translateX(0);
}

.sidebar__header {
  display: flex;
  justify-content: space-between;
  align-items: center;
  gap: 12px;
  margin-bottom: 20px;
}

.sidebar__nav {
  display: grid;
  gap: 8px;
}

.sidebar__nav button,
.sidebar__header button,
.btn-menu {
  border: 0;
  border-radius: 12px;
  padding: 11px 14px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.12);
  cursor: pointer;
}

.sidebar-overlay {
  position: fixed;
  inset: 0;
  z-index: 1100;
  background: rgba(0, 0, 0, 0.45);
  opacity: 0;
  pointer-events: none;
  transition: opacity 0.25s ease;
}

.sidebar-overlay.is-visible {
  opacity: 1;
  pointer-events: auto;
}

```

## JavaScript modular con export/import

### funciones.js - clase o funcion exportada

```

export class SidebarToggle {
  constructor(config = {}) {
    this.sidebar = document.getElementById(config.sidebarId || "sidebar");
    this.openButton = document.getElementById(config.openButtonId || "openSidebar");
    this.closeButton = document.getElementById(config.closeButtonId ||
      "closeSidebar");
    this.overlay = document.getElementById(config.overlayId || "sidebarOverlay");
    this.init();
  }

  init() {
    this.openButton?.addEventListener("click", () => this.open());
    this.closeButton?.addEventListener("click", () => this.close());
    this.overlay?.addEventListener("click", () => this.close());
    document.addEventListener("keydown", (event) => {
      if (event.key === "Escape") this.close();
    });
  }

  open() {
    this.sidebar?.classList.add("is-open");
    this.overlay?.classList.add("is-visible");
    document.body.classList.add("sidebar-open");
    this.sidebar?.setAttribute("aria-hidden", "false");
  }

  close() {
    this.sidebar?.classList.remove("is-open");
    this.overlay?.classList.remove("is-visible");
    document.body.classList.remove("sidebar-open");
    this.sidebar?.setAttribute("aria-hidden", "true");
  }
}

```

## script.js - como llamarlo

```

// script.js
import { SidebarToggle } from './funciones.js';

new SidebarToggle();

```

## JavaScript normal sin modulos

### script-normal.js - version directa

```
function initSidebarToggle() {
  const sidebar = document.getElementById("sidebar");
  const openButton = document.getElementById("openSidebar");
  const closeButton = document.getElementById("closeSidebar");
  const overlay = document.getElementById("sidebarOverlay");

  function open() {
    sidebar.classList.add("is-open");
    overlay.classList.add("is-visible");
    document.body.classList.add("sidebar-open");
    sidebar.setAttribute("aria-hidden", "false");
  }

  function close() {
    sidebar.classList.remove("is-open");
    overlay.classList.remove("is-visible");
    document.body.classList.remove("sidebar-open");
    sidebar.setAttribute("aria-hidden", "true");
  }

  openButton?.addEventListener("click", open);
  closeButton?.addEventListener("click", close);
  overlay?.addEventListener("click", close);
  document.addEventListener("keydown", (event) => {
    if (event.key === "Escape") close();
  });
}

initSidebarToggle();
```

## Variables, clases, IDs y atributos que debes cambiar

| Elemento        | Donde esta   | Como adaptarlo                                                |
|-----------------|--------------|---------------------------------------------------------------|
| #sidebar        | HTML/JS      | ID del panel lateral. Cambialo en HTML y en config.sidebarId. |
| #openSidebar    | HTML/JS      | Boton que abre. Puede estar en navbar o en cualquier parte.   |
| #closeSidebar   | HTML/JS      | Boton interno que cierra el menu.                             |
| #sidebarOverlay | HTML/JS/CSS  | Capa oscura para cerrar al hacer clic fuera.                  |
| width           | CSS .sidebar | Modifica el ancho del panel, por ejemplo 280px o 360px.       |

## Paso a paso para implementarlo en otro proyecto

1. Copia el aside, el boton de abrir y el overlay.
2. Copia CSS de .sidebar, .sidebar-overlay y body.sidebar-open.
3. Copia la clase SidebarToggle en funciones.js.
4. En script.js importa SidebarToggle y crea new SidebarToggle().
5. Prueba abrir, cerrar, cerrar con overlay y cerrar con tecla Escape.

## Errores comunes y como corregirlos

| Error                              | Causa probable                       | Solucion                                                          |
|------------------------------------|--------------------------------------|-------------------------------------------------------------------|
| El sidebar aparece siempre visible | Falta transform translateX(-105%)    | Verifica la clase .sidebar y que .is-open solo se agregue con JS. |
| No cierra con overlay              | El overlay no existe o tiene otro id | Revisa #sidebarOverlay y el constructor.                          |
| La pagina sigue haciendo scroll    | Falta body.sidebar-open              | Agrega overflow: hidden cuando el sidebar esta abierto.           |

## Checklist de prueba

- Abre desde el boton principal.
- Cierra con boton Cerrar.
- Cierra con clic en overlay.
- Cierra con Escape.
- En movil no se sale de la pantalla.

## Adaptaciones profesionales recomendadas

- Agrega focus trap si el sidebar contiene formularios.
- Usa aria-expanded en el boton de abrir para accesibilidad avanzada.
- Guarda el estado abierto/cerrado si deseas recordar el layout de un dashboard.

### 3. Submenus laterales

Los submenus laterales permiten agrupar opciones dentro del sidebar. El usuario puede abrir una categoria y ver sus opciones internas. Es clave para organizar proyectos grandes sin saturar la pantalla.

Donde se usa en paginas reales: paneles administrativos, menus de configuracion, rutas de aprendizaje, categorias de tienda, menus de usuario.

Dependencias internas: normalmente va dentro del Sidebar ocultable, pero tambien sirve en menus verticales fijos.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

| Parte      | Que debes copiar                                        | Que puedes cambiar sin romperlo                                            |
|------------|---------------------------------------------------------|----------------------------------------------------------------------------|
| HTML       | Estructura visible del componente y atributos data-*    | Textos, iconos, rutas, nombres de secciones, opciones y labels.            |
| CSS        | Clases del componente y, si aplica, variables de color. | Colores, blur, radios, espacios, sombras, tiempos de transicion.           |
| JS modular | Clase exportada con constructor e inicializacion.       | Selectores, nombres de clase activa, claves localStorage y comportamiento. |
| JS normal  | Funcion inicializadora sin import/export.               | Selectores y condiciones, igual que en la version modular.                 |

### Codigo HTML minimo

#### HTML que debes llevar a index.html

```
<aside class="sidebar" id="sidebar">
  <div class="side-group">
    <button type="button" class="side-group__toggle">
      Productos
    </button>
    <div class="side-group__content">
      <button type="button" data-section="cervezas">Cervezas</button>
      <button type="button" data-section="combos">Combos</button>
      <button type="button" data-section="promos">Promociones</button>
    </div>
  </div>

  <div class="side-group">
    <button type="button" class="side-group__toggle">
      Administracion
    </button>
    <div class="side-group__content">
      <button type="button" data-section="usuarios">Usuarios</button>
      <button type="button" data-section="reportes">Reportes</button>
    </div>
  </div>
</aside>
```

### Codigo CSS minimo

#### CSS que debes llevar a styles.css

```

.side-group {
  display: grid;
  gap: 6px;
  margin-bottom: 10px;
}

.side-group__toggle {
  width: 100%;
  display: flex;
  align-items: center;
  justify-content: space-between;
  border: 0;
  border-radius: 12px;
  padding: 12px 14px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.14);
  cursor: pointer;
}

.side-group__toggle span {
  transition: transform 0.2s ease;
}

.side-group.is-open .side-group__toggle span {
  transform: rotate(45deg);
}

.side-group__content {
  display: grid;
  gap: 6px;
  max-height: 0;
  overflow: hidden;
  transition: max-height 0.25s ease;
}

.side-group.is-open .side-group__content {
  max-height: 260px;
}

.side-group__content button {
  margin-left: 12px;
  border: 0;
  border-radius: 10px;
  padding: 10px 12px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.08);
  text-align: left;
  cursor: pointer;
}

```

## JavaScript modular con export/import

### funciones.js - clase o funcion exportada

```
export class SidebarSubmenus {
  constructor(groupSelector = ".side-group") {
    this.groups = document.querySelectorAll(groupSelector);
    this.init();
  }

  init() {
    this.groups.forEach((group) => {
      const toggle = group.querySelector(".side-group__toggle");
      toggle?.addEventListener("click", () => {
        group.classList.toggle("is-open");
      });
    });
  }
}
```

## script.js - como llamarlo

```
// script.js
import { SidebarSubmenus } from './funciones.js';

new SidebarSubmenus();
```

## JavaScript normal sin modulos

### script-normal.js - version directa

```
function initSidebarSubmenus(groupSelector = ".side-group") {
  const groups = document.querySelectorAll(groupSelector);

  groups.forEach((group) => {
    const toggle = group.querySelector(".side-group__toggle");
    toggle?.addEventListener("click", () => {
      group.classList.toggle("is-open");
    });
  });
}

initSidebarSubmenus();
```

## Variables, clases, IDs y atributos que debes cambiar

| Elemento             | Donde esta       | Como adaptarlo                                                  |
|----------------------|------------------|-----------------------------------------------------------------|
| .side-group          | HTML/CSS/JS      | Cada grupo nuevo debe usar esta clase para que JS lo reconozca. |
| .side-group__toggle  | HTML/JS          | Boton que abre y cierra el grupo.                               |
| .side-group__content | HTML/CSS         | Contenedor de opciones internas.                                |
| data-section         | Botones internos | Conecta cada opcion con una seccion del Router.                 |
| max-height           | CSS              | Aumentalo si el submenu tiene muchas opciones.                  |

## Paso a paso para implementarlo en otro proyecto

1. Inserta uno o varios .side-group dentro del sidebar.
2. Copia las reglas CSS de side-group y sus hijos.

3. Copia SidebarSubmenus a funciones.js.
4. Importa y ejecuta new SidebarSubmenus() en script.js.
5. Duplica un grupo completo para crear nuevas categorias.

## Errores comunes y como corregirlos

| Error                | Causa probable                               | Solucion                                                                        |
|----------------------|----------------------------------------------|---------------------------------------------------------------------------------|
| El submenu no abre   | Falta clase .side-group__toggle              | El boton que abre debe tener exactamente esa clase o debes cambiar el selector. |
| Se cortan opciones   | max-height muy bajo                          | Sube max-height a 400px o usa calculo dinamico con scrollHeight.                |
| Abren todos a la vez | Se esta usando una clase global mal aplicada | Asegurate de aplicar toggle solo al group actual.                               |

## Checklist de prueba

- Cada grupo abre y cierra de forma independiente.
- Las opciones internas se ven alineadas y con margen.
- La animacion no corta textos.
- Los data-section internos funcionan si estan conectados al router.

## Adaptaciones profesionales recomendadas

- Agrega modo acordeon: cerrar los otros grupos cuando uno se abre.
- Agrega iconos SVG en lugar del signo +.
- Usa aria-expanded para indicar si el grupo esta abierto.

## 4. Router de secciones

El router de secciones permite cambiar lo que se muestra en el body sin recargar la pagina. Es una base sencilla para crear una SPA pequeña: al hacer clic en un boton, se ocultan las demas secciones y se muestra solo la solicitada.

Donde se usa en paginas reales: portafolios, demos de componentes, dashboards simples, proyectos academicos, landing pages interactivas.

Dependencias internas: funciona con cualquier boton que tenga data-section y con secciones que tengan data-view-section e id coincidente.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

| Parte      | Que debes copiar                                        | Que puedes cambiar sin romperlo                                            |
|------------|---------------------------------------------------------|----------------------------------------------------------------------------|
| HTML       | Estructura visible del componente y atributos data-*.   | Textos, iconos, rutas, nombres de secciones, opciones y labels.            |
| CSS        | Clases del componente y, si aplica, variables de color. | Colores, blur, radios, espacios, sombras, tiempos de transicion.           |
| JS modular | Clase exportada con constructor e inicializacion.       | Selectores, nombres de clase activa, claves localStorage y comportamiento. |
| JS normal  | Funcion inicializadora sin import/export.               | Selectores y condiciones, igual que en la version modular.                 |

### Codigo HTML minimo

#### HTML que debes llevar a index.html

```
<nav class="section-menu">
  <button type="button" data-section="inicio">Inicio</button>
  <button type="button" data-section="servicios">Servicios</button>
  <button type="button" data-section="contacto">Contacto</button>
</nav>

<main>
  <section id="inicio" data-view-section class="view-section is-visible">
    <h2>Inicio</h2>
    <p>Contenido de inicio.</p>
  </section>

  <section id="servicios" data-view-section class="view-section">
    <h2>Servicios</h2>
    <p>Contenido de servicios.</p>
  </section>

  <section id="contacto" data-view-section class="view-section">
    <h2>Contacto</h2>
    <p>Contenido de contacto.</p>
  </section>
</main>
```

### Codigo CSS minimo

#### CSS que debes llevar a styles.css

```

.view-section {
  display: none;
  animation: fadeIn 0.25s ease;
}

.view-section.is-visible {
  display: block;
}

[data-section].is-active {
  background: #2563eb;
  color: #ffffff;
}

@keyframes fadeIn {
  from {
    opacity: 0;
    transform: translateY(8px);
  }
  to {
    opacity: 1;
    transform: translateY(0);
  }
}

```

## JavaScript modular con export/import

### funciones.js - clase o funcion exportada

```

export class SectionRouter {
  constructor(config = {}) {
    this.buttonSelector = config.buttonSelector || "[data-section]";
    this.sectionSelector = config.sectionSelector || "[data-view-section]";
    this.buttons = document.querySelectorAll(this.buttonSelector);
    this.sections = document.querySelectorAll(this.sectionSelector);
    this.defaultSection = config.defaultSection || "inicio";
    this.init();
  }

  init() {
    this.buttons.forEach((button) => {
      button.addEventListener("click", () => {
        this.show(button.dataset.section);
      });
    });

    this.show(this.defaultSection);
  }

  show(sectionId) {
    this.sections.forEach((section) => {
      section.classList.toggle("is-visible", section.id === sectionId);
    });

    this.buttons.forEach((button) => {
      button.classList.toggle("is-active", button.dataset.section === sectionId);
    });
  }
}

```

### script.js - como llamarlo

```

// script.js
import { SectionRouter } from './funciones.js';

new SectionRouter();

```

# JavaScript normal sin modulos

## script-normal.js - version directa

```
function initSectionRouter(defaultSection = "inicio") {
  const buttons = document.querySelectorAll("[data-section]");
  const sections = document.querySelectorAll("[data-view-section]");

  function show(sectionId) {
    sections.forEach((section) => {
      section.classList.toggle("is-visible", section.id === sectionId);
    });

    buttons.forEach((button) => {
      button.classList.toggle("is-active", button.dataset.section === sectionId);
    });
  }

  buttons.forEach((button) => {
    button.addEventListener("click", () => show(button.dataset.section));
  });

  show(defaultSection);
}

initSectionRouter();
```

## Variables, clases, IDs y atributos que debes cambiar

| Elemento          | Donde esta | Como adaptarlo                                                   |
|-------------------|------------|------------------------------------------------------------------|
| data-section      | Botones    | Su valor debe ser igual al id de la seccion que quieres mostrar. |
| data-view-section | Secciones  | Marca las secciones que el router puede controlar.               |
| .is-visible       | CSS/JS     | Clase que muestra la seccion activa.                             |
| .is-active        | CSS/JS     | Clase visual para el boton activo.                               |
| defaultSection    | JS         | Seccion que aparece al cargar la pagina.                         |

## Paso a paso para implementarlo en otro proyecto

1. Crea un boton con data-section='nombre'.
2. Crea una seccion con id='nombre' y data-view-section.
3. Copia CSS de .view-section y .is-visible.
4. Copia SectionRouter a funciones.js.
5. En script.js ejecuta new SectionRouter({ defaultSection: 'inicio' }).
6. Prueba que cada boton muestre solo su seccion.

## Errores comunes y como corregirlos

| Error                         | Causa probable                     | Solucion                                                        |
|-------------------------------|------------------------------------|-----------------------------------------------------------------|
| El boton no muestra nada      | data-section no coincide con el id | Iguala exactamente data-section='servicios' con id='servicios'. |
| Se ven todas las secciones    | Falta CSS display none             | Agrega .view-section { display: none; }.                        |
| La seccion inicial sale vacia | defaultSection no existe           | Usa un id real como defaultSection.                             |

## Checklist de prueba

- Cada boton muestra una seccion distinta.
- Solo una seccion queda visible a la vez.
- El boton activo cambia de estilo.
- No hay IDs repetidos en HTML.

## Adaptaciones profesionales recomendadas

- Agrega location.hash para que la URL cambie con cada seccion.
- Cierra el sidebar automaticamente al cambiar de seccion en movil.
- Carga contenido dinamico desde arrays si quieres evitar mucho HTML repetido.

## 5. Dropdown superior

El dropdown superior es un menu desplegable que aparece al hacer clic en un boton. Sirve para opciones de usuario, configuraciones rapidas, idioma, perfil, notificaciones o acciones secundarias.

Donde se usa en paginas reales: navbar de usuario, perfiles, acciones de tarjetas, menus de configuracion, dashboards y ecommerce.

Dependencias internas: puede ir dentro de la navbar fija glass o en cualquier contenedor. Usa una clase abierta para mostrar/ocultar.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

| Parte      | Que debes copiar                                        | Que puedes cambiar sin romperlo                                            |
|------------|---------------------------------------------------------|----------------------------------------------------------------------------|
| HTML       | Estructura visible del componente y atributos data-*.   | Textos, iconos, rutas, nombres de secciones, opciones y labels.            |
| CSS        | Clases del componente y, si aplica, variables de color. | Colores, blur, radios, espacios, sombras, tiempos de transicion.           |
| JS modular | Clase exportada con constructor e inicializacion.       | Selectores, nombres de clase activa, claves localStorage y comportamiento. |
| JS normal  | Funcion inicializadora sin import/export.               | Selectores y condiciones, igual que en la version modular.                 |

### Codigo HTML minimo

#### HTML que debes llevar a index.html

```
<div class="dropdown" id="userDropdown">
  <button type="button" class="dropdown__button">
    Carlos
    <span>v</span>
  </button>

  <div class="dropdown__menu">
    <button type="button" data-section="perfil">Mi perfil</button>
    <button type="button" data-section="configuracion">Configuracion</button>
    <button type="button" id="logoutButton">Cerrar sesion</button>
  </div>
</div>
```

### Codigo CSS minimo

#### CSS que debes llevar a styles.css

```

.dropdown {
  position: relative;
  display: inline-block;
}

.dropdown__button {
  border: 0;
  border-radius: 999px;
  padding: 10px 14px;
  color: #ffffff;
  background: rgba(255, 255, 255, 0.14);
  cursor: pointer;
}

.dropdown__menu {
  position: absolute;
  top: calc(100% + 10px);
  right: 0;
  min-width: 190px;
  padding: 8px;
  background: rgba(15, 23, 42, 0.88);
  border: 1px solid rgba(255, 255, 255, 0.20);
  border-radius: 16px;
  backdrop-filter: blur(18px);
  -webkit-backdrop-filter: blur(18px);
  box-shadow: 0 18px 45px rgba(0, 0, 0, 0.28);
  opacity: 0;
  transform: translateY(-8px);
  pointer-events: none;
  transition: opacity 0.2s ease, transform 0.2s ease;
}

.dropdown.is-open .dropdown__menu {
  opacity: 1;
  transform: translateY(0);
  pointer-events: auto;
}

.dropdown__menu button {
  width: 100%;
  border: 0;
  border-radius: 10px;
  padding: 10px 12px;
  color: #ffffff;
  background: transparent;
  text-align: left;
  cursor: pointer;
}

.dropdown__menu button:hover {
  background: rgba(255, 255, 255, 0.12);
}

```

## JavaScript modular con export/import

### funciones.js - clase o funcion exportada

```
export class TopDropdown {
  constructor(dropdownId = "userDropdown") {
    this.dropdown = document.getElementById(dropdownId);
    this.button = this.dropdown?.querySelector(".dropdown__button");
    this.init();
  }

  init() {
    this.button?.addEventListener("click", (event) => {
      event.stopPropagation();
      this.dropdown.classList.toggle("is-open");
    });

    document.addEventListener("click", () => this.close());
    document.addEventListener("keydown", (event) => {
      if (event.key === "Escape") this.close();
    });
  }

  close() {
    this.dropdown?.classList.remove("is-open");
  }
}
```

## script.js - como llamarlo

```
// script.js
import { TopDropdown } from './funciones.js';

new TopDropdown();
```

## JavaScript normal sin modulos

### script-normal.js - version directa

```
function initTopDropdown(dropdownId = "userDropdown") {
  const dropdown = document.getElementById(dropdownId);
  const button = dropdown?.querySelector(".dropdown__button");

  button?.addEventListener("click", (event) => {
    event.stopPropagation();
    dropdown.classList.toggle("is-open");
  });

  document.addEventListener("click", () => {
    dropdown?.classList.remove("is-open");
  });

  document.addEventListener("keydown", (event) => {
    if (event.key === "Escape") dropdown?.classList.remove("is-open");
  });
}

initTopDropdown();
```

## Variables, clases, IDs y atributos que debes cambiar

| Elemento          | Donde esta  | Como adaptarlo                                   |
|-------------------|-------------|--------------------------------------------------|
| #userDropdown     | HTML/JS     | ID del dropdown. Si cambias uno, cambia el otro. |
| .dropdown__button | HTML/JS/CSS | Boton que abre/cierra el menu.                   |

| Elemento        | Donde esta | Como adaptarlo                                                  |
|-----------------|------------|-----------------------------------------------------------------|
| .dropdown__menu | HTML/CSS   | Contenedor de opciones.                                         |
| right: 0        | CSS        | Cambia a left: 0 si quieres que abra hacia la izquierda visual. |
| data-section    | Opciones   | Puedes usarlo para abrir secciones del router.                  |

## Paso a paso para implementarlo en otro proyecto

1. Copia el contenedor .dropdown en la navbar o en una card.
2. Copia el CSS del dropdown.
3. Copia TopDropdown en funciones.js.
4. Importa y ejecuta new TopDropdown('userDropdown').
5. Agrega o elimina botones dentro de .dropdown\_\_menu segun tus opciones.

## Errores comunes y como corregirlos

| Error                           | Causa probable                           | Solucion                                                     |
|---------------------------------|------------------------------------------|--------------------------------------------------------------|
| Se abre y se cierra al instante | No se uso event.stopPropagation          | Mantén event.stopPropagation en el click del boton.          |
| Queda debajo de otros elementos | z-index bajo o padre con overflow hidden | Sube z-index o evita overflow hidden en el contenedor padre. |
| No cierra al hacer clic fuera   | Falta listener en document               | Agrega<br>document.addEventListener('click', close).         |

## Checklist de prueba

- Abre al hacer clic en el boton.
- Cierra al hacer clic fuera.
- Cierra con Escape.
- Las opciones internas son clicables.

## Adaptaciones profesionales recomendadas

- Convierte cada opcion en enlace <a> si dirige a otra pagina.
- Agrega iconos y separadores para opciones importantes.
- Cierra el dropdown despues de hacer clic en cualquier opcion interna.

## 6. Tema claro/oscuro

El cambio de tema permite alternar entre apariencia oscura y clara. En proyectos profesionales se hace con variables CSS para no reescribir todo el diseño. El tema elegido puede guardarse en localStorage para que el usuario conserve su preferencia.

Donde se usa en paginas reales: dashboards, blogs, portafolios, aplicaciones web, ecommerce y paneles de configuracion.

Dependencias internas: usa variables CSS, un atributo data-theme en <html> o <body>, un boton y localStorage.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

| Parte      | Que debes copiar                                        | Que puedes cambiar sin romperlo                                            |
|------------|---------------------------------------------------------|----------------------------------------------------------------------------|
| HTML       | Estructura visible del componente y atributos data-.*.  | Textos, iconos, rutas, nombres de secciones, opciones y labels.            |
| CSS        | Clases del componente y, si aplica, variables de color. | Colores, blur, radios, espacios, sombras, tiempos de transicion.           |
| JS modular | Clase exportada con constructor e inicializacion.       | Selectores, nombres de clase activa, claves localStorage y comportamiento. |
| JS normal  | Funcion inicializadora sin import/export.               | Selectores y condiciones, igual que en la version modular.                 |

### Codigo HTML minimo

#### HTML que debes llevar a index.html

```
<button type="button" id="themeToggle" class="theme-toggle">
  Cambiar tema
</button>
```

### Codigo CSS minimo

#### CSS que debes llevar a styles.css

```

:root {
  --page-bg: #0f172a;
  --surface: rgba(255, 255, 255, 0.12);
  --text: #ffffff;
  --muted: rgba(255, 255, 255, 0.72);
  --primary: #2563eb;
}

[data-theme="light"] {
  --page-bg: #f8fafc;
  --surface: rgba(15, 23, 42, 0.06);
  --text: #0f172a;
  --muted: #475569;
  --primary: #1d4ed8;
}

body {
  background: var(--page-bg);
  color: var(--text);
  transition: background 0.25s ease, color 0.25s ease;
}

.card,
.glass-navbar,
.sidebar {
  background: var(--surface);
}

.theme-toggle {
  border: 0;
  border-radius: 999px;
  padding: 10px 14px;
  color: #ffffff;
  background: var(--primary);
  cursor: pointer;
}

```

## JavaScript modular con export/import

### funciones.js - clase o funcion exportada

```

export class ThemeToggle {
  constructor(config = {}) {
    this.button = document.getElementById(config.buttonId || "themeToggle");
    this.storageKey = config.storageKey || "kit-theme";
    this.root = document.documentElement;
    this.init();
  }

  init() {
    const savedTheme = localStorage.getItem(this.storageKey) || "dark";
    this.apply(savedTheme);

    this.button?.addEventListener("click", () => {
      const current = this.root.dataset.theme === "light" ? "light" : "dark";
      const next = current === "light" ? "dark" : "light";
      this.apply(next);
    });
  }

  apply(theme) {
    this.root.dataset.theme = theme;
    localStorage.setItem(this.storageKey, theme);
    if (this.button) {
      this.button.textContent = theme === "light" ? "Tema oscuro" : "Tema claro";
    }
  }
}

```

## script.js - como llamarlo

```
// script.js
import { ThemeToggle } from './funciones.js';

new ThemeToggle();
```

## JavaScript normal sin modulos

### script-normal.js - version directa

```
function initThemeToggle() {
  const button = document.getElementById("themeToggle");
  const storageKey = "kit-theme";
  const root = document.documentElement;

  function apply(theme) {
    root.dataset.theme = theme;
    localStorage.setItem(storageKey, theme);
    if (button) {
      button.textContent = theme === "light" ? "Tema oscuro" : "Tema claro";
    }
  }

  const savedTheme = localStorage.getItem(storageKey) || "dark";
  apply(savedTheme);

  button?.addEventListener("click", () => {
    const current = root.dataset.theme === "light" ? "light" : "dark";
    apply(current === "light" ? "dark" : "light");
  });
}

initThemeToggle();
```

## Variables, clases, IDs y atributos que debes cambiar

| Elemento     | Donde esta  | Como adaptarlo                                                    |
|--------------|-------------|-------------------------------------------------------------------|
| #themeToggle | HTML/JS     | Boton que cambia tema. Cambia el id si actualizas el constructor. |
| --page-bg    | CSS         | Color o fondo principal de la pagina.                             |
| --surface    | CSS         | Color de tarjetas/nav/sidebar.                                    |
| storageKey   | JS          | Clave de localStorage. Cambiala si mezclas varios proyectos.      |
| data-theme   | HTML/JS/CSS | Atributo que activa las variables del tema claro u oscuro.        |

## Paso a paso para implementarlo en otro proyecto

1. Define tus colores base en :root.
2. Define los colores alternativos en [data-theme='light'] o [data-theme='dark'].
3. Usa variables en tus componentes: color: var(--text), background: var(--surface).
4. Copia ThemeToggle a funciones.js.

5. Importa y ejecuta `new ThemeToggle()`.
6. Recarga la pagina y verifica que el tema elegido se conserve.

## Errores comunes y como corregirlos

| Error                          | Causa probable                                       | Solucion                                                            |
|--------------------------------|------------------------------------------------------|---------------------------------------------------------------------|
| Al cambiar tema no cambia nada | Los componentes no usan variables CSS                | Reemplaza colores fijos por <code>var(--nombre)</code> .            |
| No recuerda el tema            | <code>localStorage</code> bloqueado o clave distinta | Verifica <code>storageKey</code> y el navegador.                    |
| Texto ilegible                 | Mismo color para fondo y texto                       | Ajusta <code>--text</code> y <code>--surface</code> en ambos temas. |

## Checklist de prueba

- El boton alterna entre tema claro y oscuro.
- Al recargar se mantiene la preferencia.
- Texto, cards y botones siguen siendo legibles.
- No quedan colores fijos que rompan el tema.

## Adaptaciones profesionales recomendadas

- Detecta preferencia del sistema con `window.matchMedia('(prefers-color-scheme: dark)')`.
- Agrega transicion suave a fondo, color y bordes.
- Usa nombres de variables semanticas: `--surface`, `--text`, `--border`, `--primary`.

## 7. Scroll progress

La barra de progreso de scroll muestra cuanto ha avanzado el usuario dentro de la pagina. Es muy util en blogs largos, documentacion, landing pages y guias extensas.

Donde se usa en paginas reales: articulos, documentacion, cursos, tutoriales, landing pages largas y paginas de lectura.

Dependencias internas: solo necesita un div fijo y escuchar el evento scroll.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

| Parte      | Que debes copiar                                        | Que puedes cambiar sin romperlo                                            |
|------------|---------------------------------------------------------|----------------------------------------------------------------------------|
| HTML       | Estructura visible del componente y atributos data-.*.  | Textos, iconos, rutas, nombres de secciones, opciones y labels.            |
| CSS        | Clases del componente y, si aplica, variables de color. | Colores, blur, radios, espacios, sombras, tiempos de transicion.           |
| JS modular | Clase exportada con constructor e inicializacion.       | Selectores, nombres de clase activa, claves localStorage y comportamiento. |
| JS normal  | Funcion inicializadora sin import/export.               | Selectores y condiciones, igual que en la version modular.                 |

### Codigo HTML minimo

#### HTML que debes llevar a index.html

```
<div class="scroll-progress" id="scrollProgress"></div>
```

### Codigo CSS minimo

#### CSS que debes llevar a styles.css

```
.scroll-progress {  
  position: fixed;  
  top: 0;  
  left: 0;  
  z-index: 2000;  
  width: 0%;  
  height: 4px;  
  background: linear-gradient(90deg, #2563eb, #7c3aed);  
  box-shadow: 0 0 16px rgba(37, 99, 235, 0.55);  
  transition: width 0.08s linear;  
}
```

### JavaScript modular con export/import

#### funciones.js - clase o funcion exportada

```
export class ScrollProgress {
  constructor(progressId = "scrollProgress") {
    this.bar = document.getElementById(progressId);
    this.init();
  }

  init() {
    window.addEventListener("scroll", () => this.update(), { passive: true });
    this.update();
  }

  update() {
    const scrollTop = window.scrollY;
    const documentHeight = document.documentElement.scrollHeight - window.innerHeight;
    const progress = documentHeight > 0 ? (scrollTop / documentHeight) * 100 : 0;

    if (this.bar) {
      this.bar.style.width = `${progress}%`;
    }
  }
}
```

## script.js - como llamarlo

```
// script.js
import { ScrollProgress } from './funciones.js';

new ScrollProgress();
```

## JavaScript normal sin modulos

### script-normal.js - version directa

```
function initScrollProgress(progressId = "scrollProgress") {
  const bar = document.getElementById(progressId);

  function update() {
    const scrollTop = window.scrollY;
    const documentHeight = document.documentElement.scrollHeight - window.innerHeight;
    const progress = documentHeight > 0 ? (scrollTop / documentHeight) * 100 : 0;
    if (bar) bar.style.width = `${progress}%`;
  }

  window.addEventListener("scroll", update, { passive: true });
  update();
}

initScrollProgress();
```

## Variables, clases, IDs y atributos que debes cambiar

| Elemento        | Donde esta | Como adaptarlo                                                 |
|-----------------|------------|----------------------------------------------------------------|
| #scrollProgress | HTML/JS    | ID de la barra. Cambialo en ambos sitios si lo renombas.       |
| height          | CSS        | Grosor de la barra. Puedes usar 3px, 4px o 6px.                |
| background      | CSS        | Color o degradado de la barra.                                 |
| z-index         | CSS        | Debe estar por encima de navbar si quieres que se vea siempre. |

| Elemento      | Donde esta | Como adaptarlo                                            |
|---------------|------------|-----------------------------------------------------------|
| passive: true | JS         | Mejora rendimiento en scroll; dejalo si solo lees scroll. |

## Paso a paso para implementarlo en otro proyecto

1. Coloca el div #scrollProgress al inicio del body.
2. Copia la regla CSS .scroll-progress.
3. Copia ScrollProgress a funciones.js.
4. Ejecuta new ScrollProgress().
5. Haz una pagina suficientemente larga y prueba el avance al hacer scroll.

## Errores comunes y como corregirlos

| Error                | Causa probable                       | Solucion                                           |
|----------------------|--------------------------------------|----------------------------------------------------|
| La barra no se mueve | La pagina no tiene altura suficiente | Agrega contenido o prueba en pagina larga.         |
| No se ve             | z-index bajo o mismo color que fondo | Sube z-index y cambia background.                  |
| Llega a mas de 100%  | Calculo de altura incorrecto         | Usa scrollHeight - innerHeight como en el ejemplo. |

## Checklist de prueba

- Al inicio la barra esta en 0%.
- Al final de la pagina llega a 100%.
- No tapa botones importantes.
- Funciona con navbar fija.

## Adaptaciones profesionales recomendadas

- Oculta la barra si la pagina no tiene scroll.
- Cambia el color de la barra segun seccion activa.
- Combinala con lectura de articulos largos o documentacion.

## 8. Back to top

El boton Back to top permite volver rapidamente al inicio de la pagina. Mejora la experiencia en paginas largas y se suele mostrar solo cuando el usuario ha bajado cierta distancia.

Donde se usa en paginas reales: blogs, documentacion, portfolios largos, tiendas, landing pages extensas y guias tecnicas.

Dependencias internas: necesita un boton fijo y detectar el scroll para mostrarlo u ocultarlo.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

| Parte      | Que debes copiar                                        | Que puedes cambiar sin romperlo                                            |
|------------|---------------------------------------------------------|----------------------------------------------------------------------------|
| HTML       | Estructura visible del componente y atributos data-*    | Textos, iconos, rutas, nombres de secciones, opciones y labels.            |
| CSS        | Clases del componente y, si aplica, variables de color. | Colores, blur, radios, espacios, sombras, tiempos de transicion.           |
| JS modular | Clase exportada con constructor e inicializacion.       | Selectores, nombres de clase activa, claves localStorage y comportamiento. |
| JS normal  | Funcion inicializadora sin import/export.               | Selectores y condiciones, igual que en la version modular.                 |

### Codigo HTML minimo

#### HTML que debes llevar a index.html

```
<button type="button" class="back-to-top" id="backToTop" aria-label="Volver arriba">
  ↑
</button>
```

### Codigo CSS minimo

#### CSS que debes llevar a styles.css

```

.back-to-top {
  position: fixed;
  right: 22px;
  bottom: 22px;
  z-index: 1300;
  width: 48px;
  height: 48px;
  border: 0;
  border-radius: 50%;
  color: #ffffff;
  background: linear-gradient(135deg, #2563eb, #7c3aed);
  box-shadow: 0 14px 35px rgba(37, 99, 235, 0.35);
  cursor: pointer;
  opacity: 0;
  transform: translateY(12px);
  pointer-events: none;
  transition: opacity 0.2s ease, transform 0.2s ease;
}

.back-to-top.is-visible {
  opacity: 1;
  transform: translateY(0);
  pointer-events: auto;
}

.back-to-top:hover {
  transform: translateY(-3px);
}

```

## JavaScript modular con export/import

### funciones.js - clase o funcion exportada

```

export class BackToTop {
  constructor(config = {}) {
    this.button = document.getElementById(config.buttonId || "backToTop");
    this.visibleAt = config.visibleAt || 360;
    this.init();
  }

  init() {
    window.addEventListener("scroll", () => this.toggle(), { passive: true });
    this.button?.addEventListener("click", () => {
      window.scrollTo({ top: 0, behavior: "smooth" });
    });
    this.toggle();
  }

  toggle() {
    const shouldShow = window.scrolly > this.visibleAt;
    this.button?.classList.toggle("is-visible", shouldShow);
  }
}

```

### script.js - como llamarlo

```

// script.js
import { BackToTop } from './funciones.js';

new BackToTop();

```

## JavaScript normal sin modulos

### script-normal.js - version directa

```
function initBackToTop(buttonId = "backToTop", visibleAt = 360) {
  const button = document.getElementById(buttonId);

  function toggle() {
    const shouldShow = window.scrollY > visibleAt;
    button?.classList.toggle("is-visible", shouldShow);
  }

  window.addEventListener("scroll", toggle, { passive: true });
  button?.addEventListener("click", () => {
    window.scrollTo({ top: 0, behavior: "smooth" });
  });

  toggle();
}

initBackToTop();
```

## Variables, clases, IDs y atributos que debes cambiar

| Elemento         | Donde esta | Como adaptarlo                                                       |
|------------------|------------|----------------------------------------------------------------------|
| #backToTop       | HTML/JS    | ID del boton. Cambialo junto con el constructor.                     |
| visibleAt        | JS         | Cantidad de pixeles bajados antes de mostrar el boton.               |
| right/bottom     | CSS        | Ubicacion del boton en pantalla.                                     |
| behavior: smooth | JS         | Hace que el regreso sea suave. Puedes quitarlo si quieres inmediato. |
| ↑                | HTML       | Puedes reemplazarlo por un icono SVG.                                |

## Paso a paso para implementarlo en otro proyecto

1. Coloca el boton al final del body.
2. Copia el CSS .back-to-top.
3. Copia BackToTop a funciones.js.
4. Importa y ejecuta new BackToTop({ visibleAt: 360 }).
5. Baja la pagina y confirma que el boton aparece.

## Errores comunes y como corregirlos

| Error                  | Causa probable                        | Solucion                                      |
|------------------------|---------------------------------------|-----------------------------------------------|
| El boton nunca aparece | Pagina muy corta o visibleAt muy alto | Baja visibleAt a 100 para probar.             |
| No sube al inicio      | El listener no carga o id incorrecto  | Revisa #backToTop y que script este al final. |
| Tapa contenido movil   | bottom/right mal ubicados             | Ajusta posicion con media queries.            |

## Checklist de prueba

- No se ve al cargar arriba.
- Aparece al hacer scroll hacia abajo.

- Al hacer clic vuelve al inicio suavemente.
- No se cruza con botones de chat o footer.

## **Adaptaciones profesionales recomendadas**

- Ocultalo cuando el usuario este en el footer si tapa contenido.
- Agrega aria-label descriptivo para lectores de pantalla.
- Usa CSS prefers-reduced-motion para usuarios que prefieren menos animaciones.

## 9. Reveal on scroll

Reveal on scroll muestra elementos con animacion cuando aparecen en pantalla. Es muy usado para que cards, secciones, imagenes y textos entren suavemente mientras el usuario navega.

Donde se usa en paginas reales: landing pages, portafolios, secciones de servicios, tarjetas de productos, galerias y storytelling.

Dependencias internas: usa la API IntersectionObserver y una clase .reveal en los elementos que quieres animar.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

| Parte      | Que debes copiar                                        | Que puedes cambiar sin romperlo                                            |
|------------|---------------------------------------------------------|----------------------------------------------------------------------------|
| HTML       | Estructura visible del componente y atributos data-*    | Textos, iconos, rutas, nombres de secciones, opciones y labels.            |
| CSS        | Clases del componente y, si aplica, variables de color. | Colores, blur, radios, espacios, sombras, tiempos de transicion.           |
| JS modular | Clase exportada con constructor e inicializacion.       | Selectores, nombres de clase activa, claves localStorage y comportamiento. |
| JS normal  | Funcion inicializadora sin import/export.               | Selectores y condiciones, igual que en la version modular.                 |

### Codigo HTML minimo

#### HTML que debes llevar a index.html

```
<section class="card reveal">
  <h2>Servicio destacado</h2>
  <p>Este bloque aparecera suavemente cuando entre en pantalla.</p>
</section>

<div class="features">
  <article class="feature-card reveal">Card 1</article>
  <article class="feature-card reveal">Card 2</article>
  <article class="feature-card reveal">Card 3</article>
</div>
```

### Codigo CSS minimo

#### CSS que debes llevar a styles.css

```

.reveal {
  opacity: 0;
  transform: translateY(22px);
  transition: opacity 0.55s ease, transform 0.55s ease;
}

.reveal.is-visible {
  opacity: 1;
  transform: translateY(0);
}

@media (prefers-reduced-motion: reduce) {
  .reveal {
    opacity: 1;
    transform: none;
    transition: none;
  }
}

```

## JavaScript modular con export/import

### funciones.js - clase o funcion exportada

```

export class RevealOnScroll {
  constructor(selector = ".reveal") {
    this.elements = document.querySelectorAll(selector);
    this.init();
  }

  init() {
    if (!('IntersectionObserver' in window)) {
      this.elements.forEach(el => el.classList.add("is-visible"));
      return;
    }

    const observer = new IntersectionObserver((entries) => {
      entries.forEach(entry => {
        if (entry.isIntersecting) {
          entry.target.classList.add("is-visible");
          observer.unobserve(entry.target);
        }
      });
    }, { threshold: 0.18 });

    this.elements.forEach(element => observer.observe(element));
  }
}

```

### script.js - como llamarlo

```

// script.js
import { RevealOnScroll } from './funciones.js';

new RevealOnScroll();

```

## JavaScript normal sin modulos

### script-normal.js - version directa

```
function initRevealOnScroll(selector = ".reveal") {
  const elements = document.querySelectorAll(selector);

  if (!('IntersectionObserver' in window)) {
    elements.forEach((el) => el.classList.add("is-visible"));
    return;
  }

  const observer = new IntersectionObserver((entries) => {
    entries.forEach((entry) => {
      if (entry.isIntersecting) {
        entry.target.classList.add("is-visible");
        observer.unobserve(entry.target);
      }
    });
  }, { threshold: 0.18 });

  elements.forEach((element) => observer.observe(element));
}

initRevealOnScroll();
```

## Variables, clases, IDs y atributos que debes cambiar

| Elemento         | Donde esta  | Como adaptarlo                                                |
|------------------|-------------|---------------------------------------------------------------|
| .reveal          | HTML/CSS/JS | Agrega esta clase a cualquier elemento que quieras animar.    |
| .is-visible      | CSS/JS      | Clase que se agrega cuando el elemento entra en pantalla.     |
| translateY(22px) | CSS         | Distancia desde donde entra. Puedes usar 12px o 40px.         |
| threshold        | JS          | Porcentaje visible necesario para activar la animacion.       |
| unobserve        | JS          | Evita repetir la animacion. Quitalo si quieres que se repita. |

## Paso a paso para implementarlo en otro proyecto

1. Agrega class='reveal' a las cards o secciones.
2. Copia CSS de .reveal y .reveal.is-visible.
3. Copia RevealOnScroll en funciones.js.
4. Ejecuta new RevealOnScroll().
5. Haz scroll y verifica que los elementos aparecen al entrar en pantalla.

## Errores comunes y como corregirlos

| Error                        | Causa probable                    | Solucion                                                                   |
|------------------------------|-----------------------------------|----------------------------------------------------------------------------|
| Los elementos nunca aparecen | JS no cargo o selector incorrecto | Revisa que tengan clase .reveal y que se ejecute el script.                |
| Aparecen de una vez          | Ya tienen .is-visible en HTML     | Deja .is-visible solo para elementos que quieras visibles desde el inicio. |
| Animacion molesta            | Transicion exagerada              | Reduce translateY y duracion.                                              |

## Checklist de prueba

- Elementos ocultos aparecen al hacer scroll.
- No quedan textos invisibles permanentemente.
- Funciona en varias secciones.
- Respeta prefers-reduced-motion.

## Adaptaciones profesionales recomendadas

- Agrega variantes: .reveal-left, .reveal-right, .reveal-scale.
- Usa delay con style='--delay: 0.2s' para animaciones escalonadas.
- No abuses: anima solo elementos importantes para no saturar la experiencia.

## 10. Contador animado

El contador animado cuenta desde cero hasta un valor objetivo cuando el elemento aparece en pantalla. Se usa mucho para indicadores, metricas, logros, estadísticas, KPIs y secciones de impacto.

Donde se usa en paginas reales: landing pages, dashboards, secciones de logros, estadísticas de marca, paneles de datos y sitios institucionales.

Dependencias internas: usa atributos data-counter y data-target; puede apoyarse en IntersectionObserver para iniciar solo cuando se vea.

Idea para reciclar: copia solo el bloque HTML que corresponde al componente, copia las reglas CSS indicadas y copia la clase JS modular o la funcion normal. Luego cambia IDs, clases y textos segun tu proyecto.

| Parte      | Que debes copiar                                        | Que puedes cambiar sin romperlo                                            |
|------------|---------------------------------------------------------|----------------------------------------------------------------------------|
| HTML       | Estructura visible del componente y atributos data-.*.  | Textos, iconos, rutas, nombres de secciones, opciones y labels.            |
| CSS        | Clases del componente y, si aplica, variables de color. | Colores, blur, radios, espacios, sombras, tiempos de transicion.           |
| JS modular | Clase exportada con constructor e inicializacion.       | Selectores, nombres de clase activa, claves localStorage y comportamiento. |
| JS normal  | Funcion inicializadora sin import/export.               | Selectores y condiciones, igual que en la version modular.                 |

### Codigo HTML minimo

#### HTML que debes llevar a index.html

```
<section class="stats">
  <article class="stat-card reveal">
    <strong data-counter data-target="1200">0</strong>
    <span>Usuarios registrados</span>
  </article>

  <article class="stat-card reveal">
    <strong data-counter data-target="98">0</strong>
    <span>Porcentaje de satisfaccion</span>
  </article>

  <article class="stat-card reveal">
    <strong data-counter data-target="50">0</strong>
    <span>Componentes reutilizables</span>
  </article>
</section>
```

### Codigo CSS minimo

#### CSS que debes llevar a styles.css

```
.stats {
  display: grid;
  grid-template-columns: repeat(auto-fit, minmax(180px, 1fr));
  gap: 16px;
}

.stat-card {
  padding: 20px;
  border-radius: 18px;
  background: rgba(255, 255, 255, 0.12);
  border: 1px solid rgba(255, 255, 255, 0.18);
  text-align: center;
}

.stat-card strong {
  display: block;
  font-size: clamp(32px, 6vw, 56px);
  line-height: 1;
  color: #ffffff;
}

.stat-card span {
  display: block;
  margin-top: 8px;
  color: rgba(255, 255, 255, 0.72);
}
```

## JavaScript modular con export/import

### funciones.js - clase o funcion exportada

```

export class AnimatedCounters {
  constructor(selector = "[data-counter]") {
    this.counters = document.querySelectorAll(selector);
    this.init();
  }

  init() {
    if (!('IntersectionObserver' in window)) {
      this.counters.forEach((counter) => this.animate(counter));
      return;
    }

    const observer = new IntersectionObserver((entries) => {
      entries.forEach((entry) => {
        if (entry.isIntersecting) {
          this.animate(entry.target);
          observer.unobserve(entry.target);
        }
      });
    }, { threshold: 0.35 });

    this.counters.forEach((counter) => observer.observe(counter));
  }

  animate(counter) {
    const target = Number(counter.dataset.target || 0);
    const duration = Number(counter.dataset.duration || 1200);
    const startTime = performance.now();

    const update = (currentTime) => {
      const elapsed = currentTime - startTime;
      const progress = Math.min(elapsed / duration, 1);
      const value = Math.floor(progress * target);
      counter.textContent = value.toLocaleString("es-CO");

      if (progress < 1) {
        requestAnimationFrame(update);
      } else {
        counter.textContent = target.toLocaleString("es-CO");
      }
    };

    requestAnimationFrame(update);
  }
}

```

## script.js - como llamarlo

```

// script.js
import { AnimatedCounters } from './funciones.js';

new AnimatedCounters();

```

## JavaScript normal sin modulos

### script-normal.js - version directa

```
function initAnimatedCounters(selector = "[data-counter]") {
  const counters = document.querySelectorAll(selector);

  function animate(counter) {
    const target = Number(counter.dataset.target || 0);
    const duration = Number(counter.dataset.duration || 1200);
    const startTime = performance.now();

    function update(currentTime) {
      const elapsed = currentTime - startTime;
      const progress = Math.min(elapsed / duration, 1);
      const value = Math.floor(progress * target);
      counter.textContent = value.toLocaleString("es-CO");

      if (progress < 1) {
        requestAnimationFrame(update);
      } else {
        counter.textContent = target.toLocaleString("es-CO");
      }
    }

    requestAnimationFrame(update);
  }

  if (!('IntersectionObserver' in window)) {
    counters.forEach(animate);
    return;
  }

  const observer = new IntersectionObserver((entries) => {
    entries.forEach((entry) => {
      if (entry.isIntersecting) {
        animate(entry.target);
        observer.unobserve(entry.target);
      }
    });
  }, { threshold: 0.35 });

  counters.forEach((counter) => observer.observe(counter));
}

initAnimatedCounters();
```

## Variables, clases, IDs y atributos que debes cambiar

| Elemento                | Donde esta | Como adaptarlo                                          |
|-------------------------|------------|---------------------------------------------------------|
| data-counter            | HTML/JS    | Marca los numeros que deben animarse.                   |
| data-target             | HTML/JS    | Numero final al que debe llegar el contador.            |
| data-duration           | HTML/JS    | Duracion opcional en milisegundos; por defecto 1200.    |
| toLocaleString('es-CO') | JS         | Formato de miles. Cambia locale si necesitas otro pais. |
| threshold               | JS         | Define que tanto debe verse el contador para iniciar.   |

## Paso a paso para implementarlo en otro proyecto

1. Crea una tarjeta o bloque con un elemento numerico.
2. Agrega data-counter y data-target='numero'.
3. Copia el CSS de .stats y .stat-card o adaptalo a tus cards existentes.
4. Copia AnimatedCounters a funciones.js.

5. Ejecuta `new AnimatedCounters()`.
6. Haz scroll hasta la seccion y verifica que cuenta hasta el valor final.

## Errores comunes y como corregirlos

| Error                   | Causa probable               | Solucion                                               |
|-------------------------|------------------------------|--------------------------------------------------------|
| Muestra NaN             | data-target no es numerico   | Usa data-target='1200' sin puntos ni comas.            |
| Cuenta demasiado rapido | duration bajo                | Agrega data-duration='2200'.                           |
| Se repite muchas veces  | No se desobserva el elemento | Mantén <code>observer.unobserve(entry.target)</code> . |

## Checklist de prueba

- Los numeros empiezan en 0.
- Llegan al valor exacto de data-target.
- La animacion inicia al entrar en pantalla.
- No aparece NaN ni valores negativos no deseados.

## Adaptaciones profesionales recomendadas

- Agrega sufijos visuales como %, + o K fuera del data-target para no romper el numero.
- Usa easing para que el contador desacelere al final.
- Conecta los valores a datos reales si luego consumes una API.

# Resumen final para reciclar las funcionalidades 1 a 10

Antes de copiar código a un proyecto real, revisa esta lista. Te ayuda a evitar los errores más comunes cuando reutilizas componentes.

| Pregunta                          | Respuesta esperada                                                        |
|-----------------------------------|---------------------------------------------------------------------------|
| ¿El HTML tiene los IDs correctos? | Cada id usado por JS existe una sola vez en la página.                    |
| ¿Las clases CSS coinciden?        | Las clases del HTML son iguales a las reglas del CSS.                     |
| ¿Los data-* coinciden?            | data-section debe coincidir con el id de la sección.                      |
| ¿Uso versión modular o normal?    | Solo una versión por página para no duplicar eventos.                     |
| ¿El componente tiene estado?      | Clases como is-open, is-visible o is-active deben ser controladas por JS. |
| ¿Se probó en móvil?               | Navbar, sidebar y dropdown no deben salirse de pantalla.                  |
| ¿Hay accesibilidad básica?        | Botones reales, aria-label cuando haga falta y cierre con Escape.         |

Con estas 10 funcionalidades ya puedes construir la base de una interfaz profesional: navegación fija, menú lateral, rutas internas, tema claro/oscuro, indicadores de scroll y animaciones de entrada.

# Actualizacion puntual 2026 - Funcionalidades 1 a 10

~~El presente documento describe las funcionalidades que se implementarán en la actualización puntual 2026 del sistema. Las funcionalidades se detallan a continuación:~~